Assignment 4

Path Planning with A* and Local Obstacle Avoidance (DWA or MPPI)

---

1 Overview

In this assignment the robot must travel from a known start point A to a known goal point B. The student first generates a global path between these two points using **A***, and then drives the robot along that path autonomously. While following the path, the robot encounters an obstacle and must avoid it using a **local planner**, which is either **DWA** (Dynamic Window Approach) or **MPPI** (Model Predictive Path Integral). The student may choose either one.

The instructor will provide A, B, and obstacle positions at lab time. The same code must work without structural changes. Write your code so that all parameters can be modified through a YAML file.

Note: The focus of this assignment is the local planner and the avoidance behavior, not A itself. A* is only used to produce a path between two given points.*

2 Inputs Provided

- A map of the workspace measuring approximately 1.5 m × 1.5 m; this map is displayed as a grid (the resolution is selected by the student). Any adjustments to the dimensions are permitted.
- Start point $A = (x_A, y_A)$ and goal point $B = (x_B, y_B)$ in the world frame.
- Obstacle position (x_o, y_o).

3 Tasks

3.1 Task 1: Global Path with A*

- Define a simple occupancy grid covering the workspace (approximately 1.5 m × 1.5 m, resolution chosen by the student, e.g. 0.05 m). The grid does not contain any obstacles at this stage.
- Use an 8-connected neighborhood so that A* can move both straight and diagonally. Diagonal motion is allowed. For convenience, the shortest route to the specified destination may be a straight path
- Run A* on this grid from A to B to produce a list of waypoints.

3.2 Task 2: Local Planner (DWA or MPPI)

Pick **one** of the two methods.

- **DWA:** sample feasible (v, ω) pairs inside the dynamic window, simulate each one for a short horizon, score them, and pick the best.
- **MPPI:** sample K noisy control sequences around a nominal one, roll each one out on the robot model, score them, and update the nominal sequence with the weighted average.

- In both cases the cost must include at least: distance to the global path, distance to the goal, and a penalty for getting close to the obstacle.
- Pre-built ROS packages of DWA or MPPI are allowed

3.3 Task 3: Obstacle Avoidance

- Inflate the obstacle by the robot radius plus a small safety margin.
- Any rollout or velocity sample that enters the inflated circle must be rejected or strongly penalized.
- The robot must reach B without touching the obstacle.

3.4 Task 4: Visualization

The student must visualize the run on screen (RViz, matplotlib, or any plotting tool). The visualization must show, at the same time:

- The global path produced by A*.
- The robot and its sensing area (the region around the robot in which it can detect obstacles).
- The local planner output: the candidate trajectories that DWA or MPPI is sampling, and the chosen one highlighted.
- The obstacle and its inflated boundary.
- The map updating as the robot moves and avoids the obstacle.

3.5 Task 5: Bonus — Unknown Obstacle

For extra credit, repeat the run with an obstacle whose position is **not** given in advance. The robot must:

- Detect the obstacle at runtime using its on-board sensor (camera or laser).
- Add the detected obstacle to its local map.
- Avoid it with the same local planner.